The Journal of Systems and Software 231 (2026) 112595

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

VulDIAC: Vulnerability detection and interpretation based on augmented CFG and causal attention learning☆

Shuailin Yang, Jiadong Ren *, Jiazheng Li, Dekai Zhang

School of Information Science and Engineering, Yanshan University, Qinhuangdao, Hebei, PR China
The Key Laboratory for Software Engineering of Hebei Province, Yanshan University, Qinhuangdao, PR China

ARTICLE INFO

Keywords:
Vulnerability detection
Control flow graph
Relation graph neural network
Causal attention learning
Interpretability

ABSTRACT

Vulnerability detection in software source code is essential for ensuring system security. Recently, deep learning methods have gained significant attention in this domain, leveraging structured information extracted from source code, and employing Graph Neural Networks (GNNs) to enhance detection performance through graph representation learning. However, conventional code graph structures exhibit limitations in capturing the comprehensive semantics of source code, and the presence of spurious features may result in incorrect correlations, which undermines the robustness and explainability of vulnerability detection models. In this paper, we propose VulDIAC, a novel framework for **Vul**nerability **D**etection and **I**nterpretation that integrates an **A**ugmented Control Flow Graph (ACFG) and a multi-task **C**ausal attention learning module based on Relational Graph Convolutional Networks, referred to as RGCN-CAL. The ACFG incorporates additional relational edges, such as reaching-define and dominator relationships, to better capture the control flow logic and data flow information within the code. The RGCN-CAL module emphasizes causal features while learning multi-relational graph representations. This approach enhances detection accuracy and provides fine-grained, line-level explanations. Experimental evaluations on two public datasets demonstrate that VulDIAC significantly outperforms baseline methods, achieving F1-Score improvements of 27.16% and 53.59%, respectively. Additionally, VulDIAC achieves better Top-k accuracy compared to LineVul on line-level vulnerability detection, which suggests its competitive performance and potential interpretability benefits.

1. Introduction

The pervasive adoption of software across diverse industries has elevated its security to a critical concern that demands rigorous attention. A significant source of security threats is the presence of software vulnerabilities, which frequently arise from defects or oversights during the software's design and development phases (Ghaffarian and Shahriari, 2017). These weaknesses provide opportunities for attackers to exploit systems, leading to significant risks such as data breaches, system failures, and severe economic losses (Liang et al., 2025). Numerous Common Vulnerabilities and Exposures (CVEs) (CVE), disclosed by platforms such as the USA National Vulnerability Database (NVD) (Booth et al., 2013), underscore the growing complexity of the cybersecurity landscape. Vulnerability analysis is becoming a key approach to mitigate threats in response to these challenges. Among the various available methods, identifying vulnerabilities in software source code has proven to be particularly effective.

Traditional static vulnerability detection tools rely on manually predefined vulnerability patterns (Lee et al., 2006) and code similarity (Jang et al., 2012; Kim et al., 2017; Li et al., 2016), while dynamic analysis methods depend on test cases and coverage (Gan et al., 2022; Pham et al., 2021). However, both approaches face significant limitations in terms of detection efficiency and accuracy. In recent years, the application of machine learning and deep learning technologies has introduced innovative research directions for source code vulnerability detection. Automated modeling approaches that leverage learning-based methods have shown substantial enhancements in detection precision (Russell et al., 2018; Li et al., 2018; Zou et al., 2021; Li et al., 2022a,b). Specifically, these methods often treat source code as flat token sequences to extract features, enhancing the ability to identify vulnerabilities. However, such approaches are inherently limited in capturing the syntactic and structural information embedded in code. Unlike natural languages, programming languages are highly structured, and governed by formally defined grammars (Allamanis et al.,

☆ Editor: Dr. Dario Di Nucci.

* Corresponding author.
E-mail addresses: shuailin.yang@hotmail.com (S. Yang), jdren@ysu.edu.cn (J. Ren), 69880353@qq.com (J. Li), linkzhangdekai@gmail.com (D. Zhang).

https://doi.org/10.1016/j.jss.2025.112595
Received 7 January 2025; Received in revised form 14 July 2025; Accepted 4 August 2025
Available online 21 August 2025

2018). To address this challenge, recent methods such as Devign (Zhou et al., 2019) and Reveal (Chakraborty et al., 2022) have leveraged composite code structure graphs extracted from Code Property Graph (CPG) (Yamaguchi et al., 2014), including Abstract Syntax Tree (AST), Control Flow Graph (CFG), Data Flow Graph (DFG), and Natural Code Sequence (NCS). By training Graph Neural Network (GNN) (Scarselli et al., 2008) on these structured representations, these methods model the structural features of source code to detect vulnerabilities more effectively. This structured representation partially mitigates the limitations in capturing syntactic and structural information of source code. However, ASTs, by incorporating a large number of nodes into the graph, inevitably introduce noise and exacerbate the problem of long-range dependencies, posing challenges to both the effectiveness and efficiency of the model. Although subsequent methods, such as AMPLE (Wen et al., 2023) and MAGNET (Wen et al., 2024), have attempted to address these issues using graph simplification techniques or heterogeneous graph modeling, they still face significant limitations in mitigating long-range dependency issues and noise interference.

Numerous studies (Li et al., 2021; Cheng et al., 2021; Zou et al., 2022; Wu et al., 2022, 2023) have leveraged subgraphs of CPG, such as Program Dependence Graph (PDG), to facilitate vulnerability detection. These approaches often utilize PDGs to capture code dependency relationships, enabling models to learn structural patterns indicative of vulnerabilities. For instance, some work (Cheng et al., 2021; Zou et al., 2022; Wu et al., 2023) focus on identifying potential vulnerability candidate nodes and applying code slicing techniques to extract localized subgraphs from PDGs. This methodology aims to reduce graph complexity, eliminate redundant information, and improve detection performance by focusing on relevant code regions. Despite their utility, PDGs have inherent limitations. While they effectively represent dependency relationships, they lack the capability to capture program control flow execution comprehensively. Alternatively, EPVD (Zhang et al., 2023) generates syntax-based CFGs from ASTs and encodes multiple representative execution paths to improve the representation of code behavior. While this enhances the representation of program execution, it is limited by incomplete path coverage, which may omit essential execution scenarios, and redundancy across paths, increasing model complexity and reducing effectiveness. Moreover, although CFGs effectively represent the execution path, they do not integrate data flow information. This separation hinders the analysis of interactions between program logic and data dependencies, which is a crucial factor in identifying vulnerabilities.

While models based on graph-structured code representations show considerable promise, their black-box nature poses significant challenges in explaining why a specific code segment is predicted to be vulnerable (Steenhoek et al., 2023; Hu et al., 2023). In particular, GNN-based methods often lack interpretability in revealing the reasoning behind their predictions. To enhance model interpretability, researchers explored fine-grained vulnerability detection techniques, such as line-level predictions, to reveal the decision-making logic behind vulnerability identification. For instance, methods like LineVul (Fu and Tantithamthavorn, 2022) rely on attention mechanisms to pinpoint vulnerable lines, capturing the features most relevant to vulnerability classification and highlighting the parts of the code the model deems significant. In addition, factual reasoning-based methods (Li et al., 2021; Cao et al., 2024) attempt to derive explanations by simplifying target graphs, such as through node removal, to isolate the underlying factors influencing the predictions. However, these methods are often susceptible to being misled by trivial features (Rahman et al., 2024), which can reduce their reliability. Since vulnerabilities frequently stem from complex causal chains, traditional data-driven deep learning models are prone to spurious correlations. For example, datasets may contain numerous non-causal feature associations that, while statistically correlated with vulnerabilities, fail to represent the fundamental triggering factors.

Motivated by the aforementioned insights, we propose VulDIAC, a novel method for function-level vulnerability detection and interpretation based on Augmented Control Flow Graphs (ACFG) and causal attention learning. Specifically, we extend traditional CFGs to construct an ACFG with four types of edges: Control Flow, Reaching-Define, Dominator, and Post-Dominator. These relations capture path dependencies and define-use features, enabling the model better to understand semantic relationships and contextual dependencies in code. We design RGCN-CAL, a multi-task causal learning module that integrates a Relational Graph Convolutional Network (RGCN) (Schlichtkrull et al., 2017) with a causal attention learning framework (Sui et al., 2022, 2024). This module models the multi-relational structure of ACFGs and utilizes fine-grained causal attention to identify relationships driving vulnerability propagation. By concentrating on causally relevant features rather than spurious correlations, it enhances both the accuracy and robustness of vulnerability detection. Furthermore, the attention weights produced by the framework serve as line-level interpretations. We conduct experiments to compare VulDIAC with several baselines (i.e., VulDeePecker (Li et al., 2018), SySeVR (Li et al., 2022b), Devign (Zhou et al., 2019), IVDetect (Li et al., 2021), AMPLE (Wen et al., 2023) and MAGNET (Wen et al., 2024)). The experimental results demonstrate that our approach outperforms the baselines in vulnerability detection and achieves better Top-k accuracy compared to LineVul (Fu and Tantithamthavorn, 2022) on line-level localization.

In summary, the key contributions of this paper are:

- We propose an Augmented Control Flow Graph (ACFG) that extends the traditional CFG by introducing multiple relational edges, enhancing the semantic representation of code.
- We develop the RGCN-CAL module, which captures the multi-relational representations of ACFG and leverages a multi-task causal attention learning framework to identify potential causal features in vulnerability propagation, enhancing vulnerability detection capabilities while providing line-level interpretations.
- We evaluate our method on real-world vulnerability datasets, showing improved performance in vulnerability detection compared to baseline methods. We released the implementation of our work at https://github.com/foresta-y/VulDIAC.

The subsequent content of this paper is arranged as follows. Section 2 describes the preliminary and motivation. Section 3 introduces our VulDIAC methodology. Sections 4 and 5 report experiments and results. Section 6 discusses our work. Section 7 is the related work. Finally, Section 8 is the conclusion of this paper.

2. Preliminary and motivation

2.1. Basic definitions

Definition 1 Control Flow Graph (CFG). (Alfred et al., 2007) The CFG of a function F is represented as a directed graph $G = (V, E)$, where $V = \{v_1, v_2, \ldots, v_n\}$ is a set of nodes, each corresponding to a distinct code block, such as a single statement or a conditional predicate. $E = \{e_1, e_2, \ldots, e_m\}$ is a set of directed edges, where each edge signifies a potential control flow between two nodes.

Definition 2 Dominator and Post-Dominator. (Ferrante et al., 1987) In a CFG, a node v_a is called a *dominator* of a node v_b if every path to v_b must first pass through v_a. In simpler terms, v_a acts as an essential checkpoint for reaching v_b. Conversely, a node v_a is called a *post-dominator* of a node v_b if every path from v_b to the program's exit must pass through v_a. This implies that v_a must be executed before the program terminates, following the execution of v_b.

Definition 3 Reaching Definition. (Alfred et al., 2007) In the context of a CFG, a *reaching definition* refers to the flow of variable assignments across nodes and edges in the graph. A definition of a variable (e.g., an assignment statement) at node v_a in the CFG is said to reach another node v_b if there exists a path from v_a to v_b, and no other definition of variable occurs along this path that would "kill" the original definition.

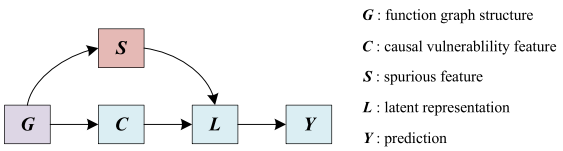

Fig. 1. A structural causal view for vulnerability detection.

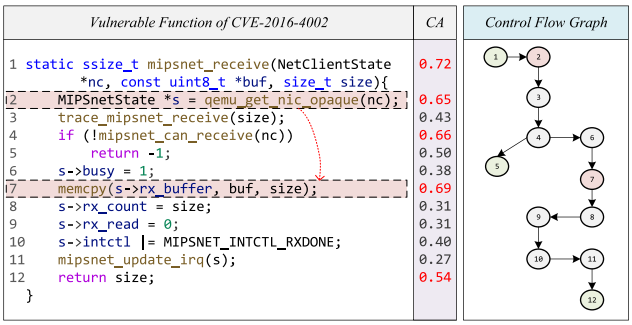

Fig. 2. A motivating example based on CVE-2016-4002.

2.2. A causal view of vulnerability detection based on code graph

We introduce a tailored Structural Causal Model (SCM) (Pearl, 2009) specifically designed for the vulnerability detection domain, capturing its unique characteristics and providing a principled framework to separate vulnerability-relevant semantics from misleading spurious patterns. As illustrated in Fig. 1, it describes the causal relationships among the following variables: function graph structure G, causal vulnerability feature C, spurious features S, latent representation L, vulnerability prediction Y. The directed arrows between the variables indicate the causal-effect relationships.

- Feature Formation ($G \rightarrow C$, S): The graph representation G of source code, such as a CFG or AST, encodes both C and S. Causal features capture semantic properties directly linked to vulnerabilities, such as unsafe API usage and control flow anomalies. In contrast, spurious features, often stemming from superficial patterns, stylistic coding conventions, incorrect code analysis, or data interpretation biases, typically represent semantics unrelated to vulnerability determination.
- Representation ($C \rightarrow L \leftarrow S$): The latent representation L, learned from the input G, integrates information from both C and S. Standard learning paradigms struggle to disentangle these features, allowing irrelevant or misleading S to influence L, ultimately compromising the model's reliability.
- Vulnerability Prediction Dependency ($L \rightarrow Y$): The prediction Y, indicating whether a program is vulnerable, is derived from the latent representation L. However, if L is dominated by spurious features S rather than causal vulnerability features C, the model may rely on non-semantic information, increasing the likelihood of errors in vulnerability detection.

In causal vulnerability detection models, the backdoor path reveals the potential interference of spurious features in the prediction process. Specifically, there exists a backdoor path between causal vulnerability features C and the prediction result Y: $C \leftarrow G \rightarrow S \rightarrow L \rightarrow Y$. Along this path, S acts as confounding variables, potentially introducing a spurious correlation between C and Y through the backdoor path. This can lead the model to make erroneous decisions based on spurious rather than causal features, affecting vulnerability prediction effectiveness.

2.3. Motivating example

In this section, we explain the rationale for introducing SCM (Pearl, 2009). We present a real-world vulnerability example, CVE-2016-4002 (CVE-2016-4002, 2016), as depicted in Fig. 2. In line 7 of the code, *buf* is copied to *rx_buffer* without verifying the size of *rx_buffer*. If the size exceeds the actual size of *rx_buffer*, a buffer overflow error occurs, potentially leading to program crashes or code execution vulnerabilities. In this example, the lines of code primarily responsible for the vulnerability are lines 2 and 7. The diagram on the right illustrates the corresponding CFG, where the nodes represent code lines. Node 1 serves as the entry point, while nodes 5 and 12 are potential exit points. The vulnerability detection model should focus on the vulnerable regions of a function when identifying vulnerabilities, specifically corresponding to the causes of these vulnerabilities, such as the code representation features of nodes 2 and 7, which correspond to causal features C. If the model erroneously focuses on features from other nodes and misclassifies the function as vulnerable, it would rely on spurious features, introducing false correlations through the backdoor path $G \rightarrow S \rightarrow L \rightarrow Y$. This would obscure the reasoning process of vulnerability detection, undermining the model's interpretability. However, deep learning models autonomously learn critical components of the input functions, specifically those code elements of high importance. Rahman et al. (2024) further highlight that state-of-the-art deep learning methods often rely on irrelevant features as proxies for vulnerability prediction, which can result in false positives. Therefore, distinguishing vulnerabilities through causal features is crucial for enhancing the effectiveness and interpretability of vulnerability detection models.

In the context of graph learning, for a graph $G = \{A, X\}$, A is the adjacency matrix, representing the connectivity between nodes, while X is the node features matrix, representing the features of each node. For complex code graph structures, different nodes and edges often vary in importance, which is crucial for capturing patterns of vulnerable code (Wen et al., 2024). Traditional GNN models often struggle to effectively focus on the important and less important parts, which may overwhelm key information (Chakraborty et al., 2022; Steenhoek et al., 2023). To address this issue, we utilize the attention mechanism (Vaswani, 2017), which dynamically learns a set of weights to guide the model in focusing on the more relevant parts of the graph when detecting vulnerabilities. We can apply soft masks to the graph structure, such as edge attention M_a, and to the node features, known as node attention M_x, where each element represents an attention weight, typically ranging from 0 to 1. The complementary mask $\overline{M} = 1 - M$ can be derived, where 1 denotes an all-one matrix. This decomposition allows the graph G to form two distinct attention graphs, $G_c = \{A \odot M_a, X \odot M_x\}$, which represents the causal attention graph, and $G_s = \{A \odot \overline{M_a}, X \odot \overline{M_x}\}$, which represents the spurious attention graph. Causal attention learning is driven by multi-task learning, enabling G_c features to contribute effectively to the prediction of classification outcomes. Firstly, to disentangle causal and spurious features at the feature representation level, we have the model predict the true labels based on G_c features to determine the presence of vulnerabilities. Meanwhile, we constrain the prediction based on G_s features to follow a uniform distribution, reducing the mutual information between spurious features and the vulnerability labels. Additionally, we introduce contrastive learning, further increasing the distance between causal and spurious features to improve their distinguishability. However, identifying effective vulnerability patterns from complex code data presents significant challenges, as trivial spurious features often interfere with the model's learning of causal features. In causal graph representation learning, the model struggles to make interventions at the data level, such as changing parts of the graph to generate a counterfactual graph (Sui et al., 2022). Therefore, we apply implicit interventions at the feature representation level of the graph. Specifically, we concatenate the G_c features with random G_s

features from the training data to form *intervention graph* features for vulnerability classification. By driving the model's predictions based on the *intervention graph* features to remain consistent during training, this method ensures that the causal features of the same function are fixed, while the spurious features vary randomly. This encourages the model to focus on learning the causal features. The causal attention mechanism will enhance model training and inference under large-scale data and intervention conditions (Sui et al., 2022, 2024), enabling the model to learn the causal attention and make decisions along the causal path $G \rightarrow C \rightarrow L \rightarrow Y$.

In real-world scenarios, however, it is often infeasible to obtain an attention graph containing only causal information. With node features as an example, we obtain the attention weights for the example function with the trained model. These attention weights are calculated from the model parameters learned during training. As illustrated in Fig. 2, we display the causal attention weights for the relevant nodes, denoted as CA, which is equivalent to the mentioned M_x. The spurious attention weights for each node are given by $[1 - CA]$. We observe that the Top-5 attention weights, highlighted in red, correspond to nodes 1, 7, 4, 2, and 12. These nodes represent the features that the model primarily attends to when detecting vulnerabilities, and they exhibit lower weights in the spurious attention graph. Although node 1 has the highest causal attention weight, the features associated with nodes 7 and 2 rank second and fourth, respectively. This means that the vulnerable features are attended by the model, even though some other nodes are also included. And these rankings provide valuable insights into the interpretability behind classifying the sample as a vulnerable function. Conversely, code features with lower causal attention weights, such as nodes 8 and 9, are less influential in the detection process. These nodes exhibit higher spurious attention weights, suggesting they are less relevant to vulnerability detection and may represent irrelevant features.

In contrast, AST-based methods (Zhou et al., 2019; Wen et al., 2023, 2024) are susceptible to issues arising from an excessive number of nodes, leading to noise and long-range dependencies. While PDG-based methods (Cheng et al., 2021; Zou et al., 2022; Wu et al., 2023) effectively represent dependencies, they lack the capacity to model the execution of program control flow. Although CFGs can mitigate the noise caused by a large number of nodes, they still face limitations related to the distance between nodes. As illustrated in Fig. 2, nodes 2 and 7 are distant in the CFG, yet they exhibit a *define-use* relationship. Consequently, we incorporate Reaching-Define edges into the CFG to alleviate this limitation. Moreover, we introduce Dominator and Post-Dominator relationships to construct an ACFG, further augmenting the graph structure. This approach significantly enhances the model's ability to express control dependencies within the code, enabling it to overcome challenges related to capturing long-distance causal chains in vulnerability detection, particularly in unforeseen scenarios.

3. Methodology

In this section, we introduce the framework of VulDIAC, as illustrated in Fig. 3, which comprises four key components. Phase I: ACFG Construction. Traditional CFGs are enhanced by incorporating dominator, post-dominator, and reaching-define edges to construct an ACFG, which improves the representation of control dependencies and data flows. Phase II: Feature Representation. Semantic and contextual information from the ACFG is encoded using a pre-trained model, while edge types are also encoded to maintain the crucial structural distinctions. Phase III: Causal Vulnerability Detection Model. The processed features are fed into the RGCN-CAL model, which is trained with multiple loss functions to achieve effective feature separation and robust representation learning. This enables the model to discern causal features while mitigating the impact of spurious ones. Phase IV: Interpretation. A causal attention mechanism is employed to provide interpretable results by highlighting the Top-k vulnerable lines in the code.

3.1. ACFG construction

As shown in Fig. 4, following previous studies (Li et al., 2018, 2022b), we transform functions into abstract representations to standardize code expressions while preserving their functionality. Specifically: (1) removing non-ASCII characters and comments, (2) normalizing variable names to $var\#$, and (3) normalizing function names to $func\#$ while retaining keywords and API library functions, where # represents a numerical identifier. This approach reduces the influence of irrelevant features introduced by coding styles, compresses the vocabulary, and focuses on the core logic and functionality of the code.

For a given function code snippet, the CFG serves as the foundation for analysis, as illustrated in the left diagram of Fig. 5. However, traditional CFGs primarily depict the basic execution paths of a program, focusing solely on the control flow order between nodes. This makes it difficult to capture deeper logical constraints and variable dependencies, which may limit the accuracy of analysis in complex vulnerability scenarios.

As shown in Fig. 5, we extend the traditional CFG by introducing several relations based on the definitions provided in Section 2.1, constructing the ACFG.

- *Control Flow Edge*: Control flow edges describe the basic execution paths of a program, represented by black solid lines in the figure. These edges form the foundation of the CFG, capturing the sequential and branching execution structure of the function.
- *Dominator Edge*: A dominator edge defines the execution prerequisite of a node, meaning every path leading to a specific node must pass through its dominator. For example, node 2 dominates node 3 and node 4, while node 3 dominates node 4. But we only add a dominator edge from node 2 to node 3 because node 3 is the closest node to node 2. The dominator edge from node 2 to node 4 is removed.
- *Post-Dominator Edge*: A post-dominator edge defines the execution consequence of a node, meaning every path exiting a specific node must pass through its post-dominator. Similarly, we introduce direct post-dominator edges, analogous to a post-dominator tree. For example, node 9 post-dominates nodes 7 and 8, but we only establish a post-dominator edge from node 9 to node 8 as its nearest post-dominator node, and the edge from node 9 to node 7 is removed.
- *Reaching-Define Edge*: Reaching-define edges capture the relationship between variable definitions and their subsequent usages. We add all definition-use edges. As shown in Fig. 5, structure variable *var6* is defined in node 2. If the conditional check result of line 4 is *false*, its definition *var6->var8* can reach node 7, so there is a reaching-define edge between them. Node 6 defines *var6->var7* instead of using it, so no reaching-define edge is added between them.

Incorporating dominator and post-dominator edges into the CFG significantly enhances its ability to represent execution constraints and subsequent paths within a function. Dominator relationships are particularly useful in conditional branching scenarios, as they identify whether specific nodes are governed by upstream conditional nodes, facilitating the detection of code paths with missing or incorrect logical constraints. Conversely, post-dominator edges capture successor relationships in the control flow, offering critical insights for identifying potential exception recovery points and termination conditions. Furthermore, reaching-define edges strengthen data flow analysis by effectively tracking variable operation chains associated with issues like uninitialized variables, insecure data propagation, and variable overwrites.

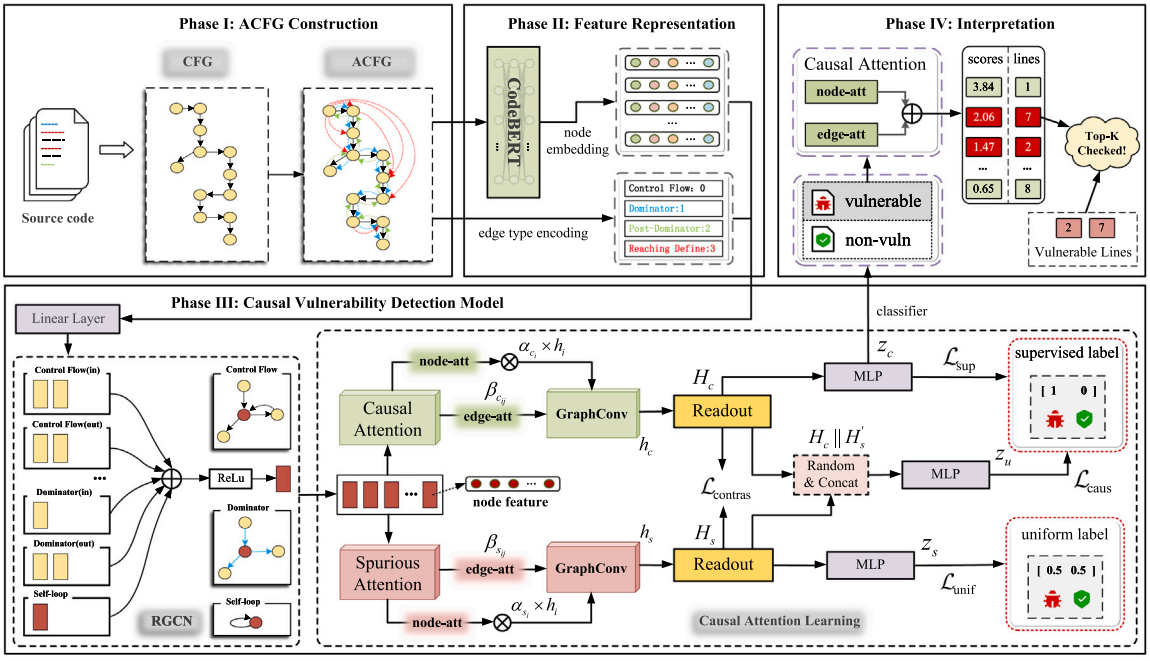

Fig. 3. A comprehensive overview of VulDIAC framework.

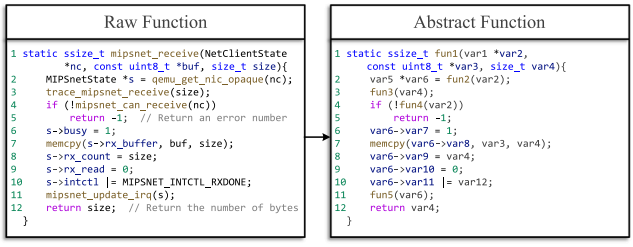

Fig. 4. An example of transforming the raw function into an abstract function.

3.2. Feature representation

As Phase II shown in Fig. 3, to enable effective program modeling and analysis using GNNs (Scarselli et al., 2008), we represent the nodes and edges of the ACFG in a structured manner that captures both semantic and structural information. This involves embedding node features and encoding edge semantics to maximize the GNN's ability to learn program properties.

Node Embedding. In the ACFG, nodes represent individual code statements or code blocks. To generate high-quality embeddings that capture both syntax and semantics, we utilize CodeBERT (Feng et al., 2020), which was pre-trained on a robustly optimized RoBERTa (Liu et al., 2019) model using a 20 GB code corpus. Similar studies have demonstrated CodeBERT's effectiveness in software vulnerability detection tasks (Hin et al., 2022), Fu and Tantithamthavorn (2022). Furthermore, it is a pre-trained bimodal model that learns from both source code and its natural language descriptions, allowing it to capture a broader vocabulary and better understand the logical semantics of the code.

Edge Type Encoding. The edges in an ACFG represent a diverse set of relationships, including control flow, dominator, post-dominator, and reaching-define dependencies. Each edge type encapsulates unique semantic information, which is essential for comprehending the program's behavior. To encode this diversity, we assign unique identifiers to each edge type as {CFG: 0, DOMINATOR: 1, POST_DOMINATOR: 2, REACHING_DEF: 3}. These identifiers are used to condition the GNN's message-passing mechanism, allowing it to apply type-specific transformation matrices during training and inference. For example, control flow edges emphasize the dependencies in the program's execution order, while reaching-define edges focus on the paths between variable definitions and their usages.

By integrating these edge types into the message-passing framework, the GNN is empowered to learn nuanced relationships between nodes, enriching its ability to model complex program semantics.

3.3. Causal vulnerability detection model

In this paper, we address vulnerability detection from a causal perspective. Specifically, the input function features can be categorized into spurious features, which hinder the model's discriminative ability, and causal features, which facilitate accurate decision-making, as discussed in Section 2.2. Inspired by causal attention learning (Sui et al., 2022, 2024), our goal is to disentangle the causal features from the input features and enable the model to rely on these causal features for determining whether a function contains vulnerabilities.

Formally, given a function ACFG $= \{V, E, X\}$, where V denotes the set of nodes in the graph corresponding to basic blocks or code statements in the program, X represents the feature matrix where each row corresponds to a node feature vector extracted as embeddings from CodeBERT (Feng et al., 2020), and E is the set of edges that describe relationships between nodes, including control flow, dominator, post-dominator and reaching-define. By leveraging an attention mechanism, we decompose the node features into causal features and spurious features, which are processed independently through dedicated GNN layers. Subsequently, we propose a multi-task learning framework to utilize the separated features fully. Specifically, as illustrated in Fig. 3 Phase III, the RGCN-CAL module is composed of the following steps.

Step I Dimension reduction. As a result of the high dimensionality of CodeBERT (Feng et al., 2020) embeddings (768), the computational burden on the graph neural network is significantly increased. To address this issue, we employ a linear layer to map the node features to a lower-dimensional space.

$$h = \sigma(W_r X). \tag{1}$$

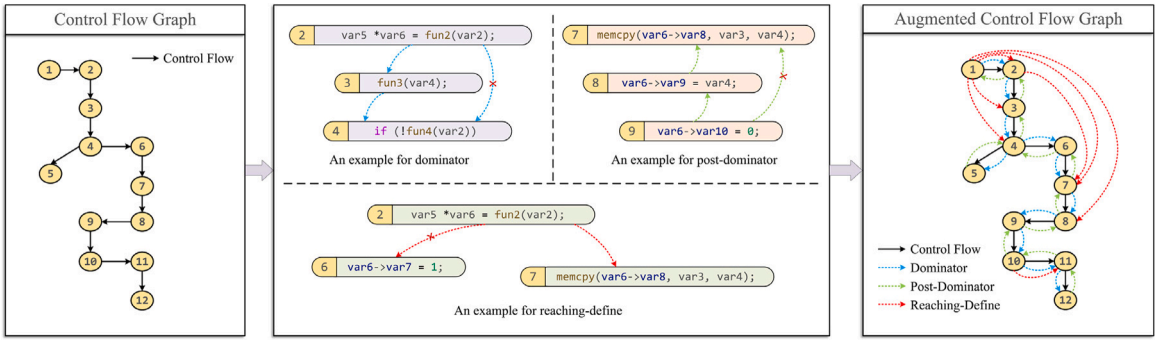

Fig. 5. A running example for constructing an Augmented Control Flow Graph (ACFG) based on the code snippet of CVE-2016-4002.

Let $X \in \mathbb{R}^{N \times d_{\text{in}}}$ represent the original node feature matrix, where N is the number of nodes; $W_r \in \mathbb{R}^{d \times d_{\text{in}}}$ is the learnable parameter matrix, and $\sigma(\cdot)$ is the non-linear activation function.

Step II Model relationships. In this step, we employ RGCN (Schlichtkrull et al., 2017) to handle multiple types of relationships in the graph. Each type of relationship updates the node features differently from its neighboring nodes. RGCN considers the various existing relationships while updating the node features. The module uses both node features and edge features (e.g., in, out, edge types). The edge features represent the relationships between nodes in the graph, and each type of edge corresponds to a different transformation matrix in the RGCN. The propagation rule is defined as:

$$h^{(l+1)} = \sigma\left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)} + W_0^{(l)} h_i^{(l)}\right). \tag{2}$$

Where $h^{(l)} \in \mathbb{R}^{d \times d}$ represents the features of node i at the lth layer, and d denotes the dimension of the hidden layer. $\mathcal{R}$ is the set of relationship types, $\mathcal{N}_i^r$ denotes the set of neighboring nodes of node i under relationship r, and $h_j^{(l)}$ is the features of the node j that is adjacent to node i under relationship r. $W_r^{(l)}$ is the learnable weight matrix corresponding to relationship r, $c_{i,r}$ is a normalization coefficient, typically set as the degree of the node, and $W_0^{(l)}$ is the learnable weight matrix for self-loops.

Step III Causal Attention Learning. To distinguish between causal and spurious features, we apply attention-based soft masks to both node-level and edge-level features. For each node i, the attention weights for its causal and spurious features are computed as follows:

$$\alpha_{c_i}, \alpha_{s_i} = \text{softmax}\left(W_{\text{node}} h_i\right). \tag{3}$$

$$\beta_{c_{ij}}, \beta_{s_{ij}} = \text{softmax}\left(W_{\text{edge}}[h_i \parallel h_j]\right). \tag{4}$$

Where $h_i \in \mathbb{R}^d$ represents the hidden representation of node i, and $[h_i \parallel h_j] \in \mathbb{R}^{2d}$ denotes the concatenation of features from node i and node j. $W_{\text{node}} \in \mathbb{R}^{2 \times d}$ and $W_{\text{edge}} \in \mathbb{R}^{2 \times 2d}$ are learnable parameter matrices. α_{c_i} and $\beta_{c_{ij}}$ represent the attention weights for causal features, while α_{s_i} and $\beta_{s_{ij}}$ represent the attention weights for spurious features. It is noting that $\alpha_{c_i} + \alpha_{s_i} = 1$ and $\beta_{c_{ij}} + \beta_{s_{ij}} = 1$. As described in Section 2.3, this study aims to assign higher values of α_{c_i} or $\beta_{c_{ij}}$ to nodes or edges that are more closely associated with potential vulnerabilities, indicating that these parts are causal features. Conversely, nodes or edges with smaller α_{c_i} or $\beta_{c_{ij}}$ values are associated with spurious parts of the graph, and their corresponding α_{s_i} or $\beta_{s_{ij}}$ values are larger. This distinction will be achieved through the training loss. The features derived from causal attention will be used for vulnerability detection tasks, while the features derived from spurious attention will be used to predict a uniform distribution, suggesting their lower relevance to vulnerability detection. Through the multi-task loss introduced in step IV, the model is encouraged to implicitly learn such patterns from the data, allowing it to place emphasis on causal features when determining whether a function is vulnerable. We can obtain the node-attention-based causal graph features and spurious graph features by the Eq. (5).

$$h_{c,i} = \alpha_{c_i} h_i; h_{s,i} = \alpha_{s_i} h_i. \tag{5}$$

Next, we apply edge attention and input the features into separate graph convolutional networks for further extraction.

$$h_c^{(l+1)} = \sigma\left(\sum_{(i,j) \in E} \beta_{c_{ij}} W_c^{(l)} h_{c,j}^{(l)}\right). \tag{6}$$

$$h_s^{(l+1)} = \sigma\left(\sum_{(i,j) \in E} \beta_{s_{ij}} W_s^{(l)} h_{s,j}^{(l)}\right). \tag{7}$$

Where $W_c^{(l)}$ and $W_s^{(l)}$ represent the weight matrices for the convolutional layers in the corresponding branches. We then use a pooling operation to extract the causal and spurious features representing the graph.

$$H_c = \phi(\{h_{c,i} | i \in V\}); H_s = \phi(\{h_{s,i} | i \in V\}). \tag{8}$$

Where ϕ is a pooling function, such as sum or mean. Then, to map causal features and spurious features to the final learning task separately, we employ distinct Multi-Layer Perceptrons (MLP) (Suter, 1990) branches to generate the outputs.

$$z_c = MLP_c(H_c), z_s = MLP_s(H_s). \tag{9}$$

Here, z_c represents the causal branch, while z_s represents the spurious branch.

Step IV Multi-task loss functions. We use multi-task loss functions to guide the model in distinguishing causal features from spurious ones. The combined effect of multiple losses helps the model learn causal patterns from the vulnerable code.

Disentanglement. To ensure that the model predicts vulnerabilities based solely on causal features, we define a cross-entropy loss function for classification, where the ground-truth labels (e.g., 0 for non-vulnerable and 1 for vulnerable) are used as supervision.

$$\mathcal{L}_{\text{sup}} = -\mathbb{E}_D[y^T log(z_c)]. \tag{10}$$

Where $\mathbb{E}_D$ denotes the mathematical expectation over the dataset D. Conversely, spurious features should not contribute to classification. To enforce this, we minimize the KL divergence to constrain the predictions based on spurious features z_s,

$$\mathcal{L}_{\text{unif}} = \mathbb{E}_D[KL(y_{\text{unif}}, z_s)]. \tag{11}$$

Specifically, the KL divergence measures the difference between two probability distributions. In this paper, we aim to constrain the predictions z_s based on spurious features to approximate a uniform distribution y_{unif} (e.g., 0.5 for non-vulnerable and 0.5 for vulnerable). This reduces the mutual information between spurious features and the class labels. Optimizing the two objectives effectively guides the model's learning process, particularly with respect to node and edge

attention. This enables the disentanglement of causal and spurious features, encouraging that the model's predictions rely more on causal features.

Do-calculus. As mentioned in Section 2.2, backdoor paths may exist in causal vulnerability detection models, causing predictions to rely on spurious features. Although we have separated causal and spurious features using attention mechanism, the above approach still cannot ensure that vulnerability predictions consistently depend on causal features. This limitation arises as a result of trivial patterns or noise, which lead to shifts in the learned attention caused by the confounding effects of spurious features. A promising solution is backdoor adjustment (Pearl, 2014), which involves stratifying confounders and pairing the target causal graph with each spurious graph to form *intervention graphs* through factual or counterfactual inference to derive the causal graph. However, cause of the irregular nature of graph data, it is impractical to perform interventions at the data level, such as modifying parts of a graph to generate counterfactual graph data. To address this challenge, we simulate the concept of *do-calculus* (Pearl, 2014) by intervening on causal features, guiding the model to distinguish stable causal relationships from unstable spurious ones. This is achieved using the following loss function, which is guided by backdoor adjustment (Sui et al., 2022):

$$z_u = MLP_u(H_c \,\|\, H_s'). \tag{12}$$

$$\mathcal{L}_{\text{causal}} = -\mathbb{E}_D[\sum_{s\in\mathcal{T}} y^T log(z_u)]. \tag{13}$$

z_u represents the prediction obtained from the classifier based on the joint representation of causal features and the randomized spurious features H_s'. $\mathcal{T}$ denotes the stratified set of estimated spurious features, which is collected from the training data.

Contrastive. We introduce contrastive learning to further assist the model in enhancing the distinctiveness and stability of feature representations, building upon the disentanglement of causal and spurious features.

$$\mathcal{L}_{\text{contra}} = \mathbb{E}_D[\max(0, margin - \delta(H_c, H_s))]. \tag{14}$$

Where δ represents the distance calculation. The optimization objective of this loss is to ensure that H_c and H_s are as disentangled as possible, specifically by maintaining a separation greater than the defined margin.

Finally, the objective of VulDIAC can be defined as the total loss:

$$\mathcal{L} = \mathcal{L}_{\text{sup}} + \lambda_1\mathcal{L}_{\text{unif}} + \lambda_2\mathcal{L}_{\text{causal}} + \lambda_3\mathcal{L}_{\text{contra}}. \tag{15}$$

where λ_1, λ_2, and λ_3 are hyperparameters that determine the relative importance of different learning objectives. Notably, the final vulnerability prediction of the VulDIAC model is determined by z_c.

3.4. Interpretation

We assess whether causal learning relies on vulnerability patterns to effectively predict vulnerable functions by designing an interpretable method. We employ the causal attention mechanism within the VulDIAC to identify vulnerable lines of code for functions predicted as vulnerable. Typically, a vulnerability score is calculated for each line of code, and lines with higher scores are considered high-risk areas for potential vulnerabilities (Li et al., 2021; Fu and Tantithamthavorn, 2022). If the actual vulnerable lines achieve high rankings, this indicates that they play a substantial role in the prediction. Consequently, the causal features appear to encode meaningful vulnerability patterns, thereby supporting the effectiveness of causal learning. The final score for a node i is determined by jointly considering the attention weight α_i of the node itself and the attention weights of the edges associated with that node. This process can be represented as Eq. (16).

$$\alpha_i = \alpha_{c_i} + \frac{1}{2}\left(\sum_{j\in N(i)} \beta_{c_{ji}} + \sum_{j\in N(i)} \beta_{c_{ij}}\right). \tag{16}$$

Table 1
The statistical characteristics of datasets.

Dataset	Vulnerable	Non-vulnerable	Total	Ratio
FFmpeg+Qemu	12,411	14,826	27,237	1:1.19
Big-Vul	5,562	98,167	103,729	1:17.65

The formula indicates that the attention score of a node is computed by aggregating its own attention weight with the edge attention weights through symmetric propagation between the source node and the target node. Subsequently, the node scores are mapped to their corresponding lines of code, and the line scores are ranked in descending order.

4. Experimental setup

4.1. Implementation

Our experiments were conducted on an NVIDIA A40 GPU server running Ubuntu 22.04 with CUDA 12.4, equipped with an Intel(R) Xeon(R) Gold 6348 CPU @ 2.60 GHz and 128 GB of RAM. The VulDIAC framework leverages PyTorch and DGL to implement graph-based neural network modules, while the CFGs of source code are extracted using the robust code analysis tool Joern (Yamaguchi et al., 2014). For code embedding, we utilized CodeBERT (Feng et al., 2020). Each dataset was split into training, validation, and testing sets in an 8:1:1 ratio during model training. We set the number of epochs to 100, the batch size to 64, and the learning rate to 0.001. For the multi-task loss, the values of λ_1, λ_2 and λ_3 were all set to 0.5. The best-performing model, determined by the highest F1-Score on the validation set, was subsequently used for testing.

4.2. Datasets

In this study, we utilized two prominent datasets, FFmpeg+Qemu (Zhou et al., 2019) and Big-Vul (Fan et al., 2020), to evaluate the performance of our method in software vulnerability detection. Both datasets are derived from real-world programs and provide ground truth labels for vulnerable and non-vulnerable functions. The statistical details of these datasets are presented in Table 1. The FFmpeg+Qemu dataset is balanced and consists of function-level code snippets extracted from two widely used open-source projects, FFmpeg (FFmpeg) and Qemu (Qemu). Conversely, the Big-Vul dataset is a large-scale, imbalanced dataset comprising code snippets associated with documented Common Vulnerabilities and Exposures (CVE) (CVE) records. It offers a diverse and comprehensive representation of vulnerabilities across multiple open-source projects.

4.3. Evaluation metrics

Following prior works (Li et al., 2018; Zhou et al., 2019; Wen et al., 2023, 2024), We use four commonly adopted metrics to evaluate the binary classification performance of function-level vulnerability detection: Accuracy, Precision, Recall (also known as True Positive Rate, TPR), and F1-Score. The corresponding formulas are provided below.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{17}$$

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{18}$$

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{19}$$

$$\text{F1} - \text{Score} = \frac{2 \cdot P \cdot R}{P + R}. \tag{20}$$

In these metrics, True Positives (TP) refer to correctly identified vulnerable functions, False Positives (FP) are non-vulnerable functions

Table 2
Experimental results of VulDIAC and other baselines on three datasets.

Baselines	FFmpeg+Qemu				Big-Vul			
	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
VulDeePecker	51.20	47.39	40.17	43.48	68.45	22.19	14.35	17.43
SySeVR	49.86	46.14	55.82	50.52	72.67	30.34	18.21	22.76
Devign	58.15	52.84	61.29	56.76	81.83	28.44	35.40	31.54
IVDetect	59.36	54.40	66.56	59.87	85.37	30.04	38.08	33.59
AMPLE	60.30	54.23	74.66	62.83	85.76	34.53	32.88	33.68
MAGNET	66.75	57.42	76.31	65.53	87.04	30.90	38.43	34.26
VulDIAC	**83.47**	**78.84**	**88.37**	**83.33**	**95.99**	**65.07**	**44.17**	**52.62**

misclassified as vulnerable, True Negatives (TN) are correctly identified non-vulnerable functions, and False Negatives (FN) are vulnerable functions incorrectly classified as non-vulnerable. We rank vulnerability scores in descending order for line-level vulnerability interpretation, evaluating detection performance using Top-k accuracy, which measures the percentage of vulnerable functions with at least one actual vulnerable line ranked within the top k.

4.4. Research questions

RQ1: How does VulDIAC perform on function-level vulnerability detection?

RQ2: What is the impact of different relationships in ACFG on the model's vulnerability detection performance?

RQ3: How do model variants affect the performance of VulDIAC? RQ3.1: How do various graph neural networks perform in capturing and learning from the complex relational structure of ACFGs? RQ3.2: How does the choice of loss function influence model training and the effectiveness of vulnerability detection?

RQ4: Can KL divergence effectively constrain the impact of spurious features on vulnerability detection?

RQ5: How does VulDIAC perform on vulnerability line-level localization?

5. Experimental results

5.1. Performance of VulDIAC

In this section, we compare a series of state-of-the-art baseline models, including token-based methods (e.g., VulDeePecker (Li et al., 2018) and SySeVR (Li et al., 2022b)) and graph-based methods (e.g., Devign (Zhou et al., 2019), IVDetect (Li et al., 2021), AMPLE (Wen et al., 2023), and MAGNET (Wen et al., 2024)). The experimental results presented in Table 2. Compared to other baseline models on the FFmpeg+Qemu (Zhou et al., 2019) and Big-Vul (Fan et al., 2020) datasets, VulDIAC achieves significant improvements across all evaluation metrics, highlighting its leading position in this task.

On the FFmpeg+Qemu dataset, VulDIAC performs exceptionally well. Compared to the best-performing baseline model, MAGNET, it achieves increases of 25.05%, 37.30%, and 27.16% in accuracy, precision, and F1-Score, respectively, showcasing its remarkable ability for high-precision vulnerability classification. Furthermore, VulDIAC achieves the highest recall among all models, demonstrating its ability to comprehensively identify vulnerability instances and effectively address other models' limitations in handling complex vulnerability patterns. On the imbalanced Big-Vul dataset, VulDIAC demonstrates exceptional robustness and adaptability. It surpasses the second-best model, MAGNET, in accuracy by nearly 9%, achieving a significant margin over other baselines as well. However, due to the dataset's inherent imbalance, the improvements in precision and recall are relatively moderate. Nevertheless, its F1-Score improves by 53.59% compared to the second-best model, demonstrating the effectiveness of VulDIAC in handling imbalanced datasets. These results demonstrate VulDIAC's ability to balance real vulnerability detection and false positive minimization effectively.

It is worth noting that traditional token-based models (e.g., VulDeePecker and SySeVR) exhibit F1-Score below 50% on both datasets due to their lack of capability to model the structural and semantic features of code, revealing their limitations when dealing with complex vulnerability patterns. Alternatively, graph-based methods (e.g., AMPLE and MAGNET) demonstrate improved performance by leveraging the structural and semantic information of CPGs, but still fall short compared to VulDIAC, particularly in terms of recall and F1-Score. The exceptional performance of VulDIAC can be attributed to its unique multi-task architecture design. By leveraging RGCN to learn semantic relationship features, such as control flow, dominator relationships, and define-use information from ACFGs, VulDIAC can capture fine-grained vulnerability patterns. Additionally, by disentangling causal and spurious features, the model can concentrate more effectively on causally relevant vulnerability features, which leads to improved detection accuracy and a more comprehensive analysis. This advantage is particularly evident in its consistently high recall, which highlights VulDIAC's ability to identify a wide range of vulnerabilities with precision and thoroughness.

> **Answer to RQ1:** VulDIAC outperforms all baselines in precision, recall, and F1-Score, achieving notable improvements in F1-Score with a 27.16% increase on the FFmpeg+Qemu dataset and a 53.59% boost on the Big-Vul dataset.

5.2. Effect of ACFG relationships

In this section, we explore the impact of different edge relationship types in ACFGs on vulnerability detection performance. To achieve this, we design three model variants for comparative experiments: (1) without any augmented edge relationships, retaining only the basic control flow (indicated as *w/o Augment*); (2) without dominator and post-dominator relationships, while keeping reaching-define relationships (indicated as *w/o D & PD*); and (3) without reaching-define relationships, while retaining dominator and post-dominator relationships (indicated as *w/o RD*). The experimental results are summarized in Table 3.

The results demonstrate that the *w/o Augment* exhibits the weakest F1-Score and recall across both datasets. However, its precision remains competitive on the FFmpeg+Qemu dataset compared to configurations with augmented edges. This implies that while it detects fewer vulnerable instances, it achieves lower false positive rates, suggesting that basic control flow features alone can reliably identify non-vulnerable patterns in balanced scenarios. In contrast, its drastic drop in precision on the imbalanced dataset Big-Vul reveals that without augmented relations, the model has difficulty distinguishing subtle vulnerability patterns from the vast majority of benign codes, leading to increased false positives in real-world data-skew scenarios. It is noteworthy that on the FFmpeg+Qemu dataset, although there is a slight reduction in precision compared to the w/o Augment, the introduction of augmented edge relationships significantly improves model performance. For instance, on the FFmpeg+Qemu dataset, the *w/o D & PD* improves the F1-Score and recall by 7.30% and 17.30%, respectively, compared

Table 3
Evaluation of the impact of different types of relations in ACFG on vulnerability detection.

Method	FFmpeg+Qemu				Big-Vul			
	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
w/o Augment	79.25	**80.63**	71.68	75.89	87.74	43.00	31.44	36.33
w/o D & PD	82.08	78.97	84.05	81.43	94.77	50.53	43.69	46.86
w/o RD	81.26	80.29	79.47	79.87	94.04	43.34	42.23	42.78
VulDIAC	**83.47**	78.84	**88.37**	**83.33**	**95.99**	**65.07**	**44.17**	**52.62**

to the *w/o Augment*. Similarly, the *w/o RD* achieves improvements of 5.24% and 10.87% in F1-Score and recall, respectively. These results demonstrate that augmented edge relationships, such as dominators, post-dominators, and reaching definitions, enhance the model's capacity to effectively capture complex vulnerability patterns, including path context and data flow information, which improves the ability to identify vulnerabilities.

Specifically, the reaching-define relationship contributes significantly to the overall performance of the model. When without this relationship (*w/o RD*), the recall decreases by 8.9 percentage points and the F1-Score drops by 3.46 percentage points compared to the VulDIAC. This reduction highlights its importance in capturing dependencies between variable definitions and their usage. Without the dominator and post-dominator relationships (*w/o D & PD*), the recall declines from 88.37% to 84.05%, demonstrating their essential role in identifying crucial control flow paths. On the more challenging Big-Vul dataset, the inclusion of augmented edge relationships similarly leads to significant performance improvements. The *w/o D & PD* and *w/o RD* achieve F1-Score of 46.86% and 42.78%, respectively, further validating the crucial role of augmented edge relationships in handling complex and imbalanced datasets. Ultimately, VulDIAC outperforms all other variants in terms of overall performance on both datasets.

> **Answer to RQ2:** The augmented edge relationships in ACFG are pivotal for boosting vulnerability detection, and their absence results in a significant performance drop.

5.3. Ablation study on model variants

In this section, we conducted an ablation study on the balanced dataset FFmpeg+Qemu (Zhou et al., 2019) and imbalanced dataset Big-Vul (Fan et al., 2020) to evaluate the impact of different model variants on VulDIAC.

5.3.1. Performance analysis of GNN variants

This section presents a performance comparison of RGCN-CAL module variants and analyzes the different GNNs in learning from complex relational ACFGs. As illustrated in Fig. 6, the RGCN-based model consistently achieves higher recall and F1-Score than the other models across both datasets. This advantage likely stems from RGCN's inherent ability to model diverse relations explicitly. Such capability allows the model to distinguish and leverage the contextual information provided by different relations, enabling a more nuanced understanding of vulnerability semantics. In particular, the semantic meanings of edge types play a crucial role in determining node embeddings and the overall graph representation.

In contrast, GGNN (Li et al., 2015) used for vulnerability detection in previous studies (Zhou et al., 2019; Chakraborty et al., 2022; Zou et al., 2022) demonstrates competitive results, achieving the best precision on the FFmpeg+Qemu dataset. This can be attributed to its gated mechanism for sequential information propagation. However, due to its inability to explicitly model multiple types of relationships, its effectiveness in leveraging the complex relational structures of ACFGs may be limited, making it inferior to RGCN, especially on the Big-Vul dataset. Similarly, GAT (Veličković et al., 2017) demonstrates strength in prioritizing important nodes through its self-attention mechanism.

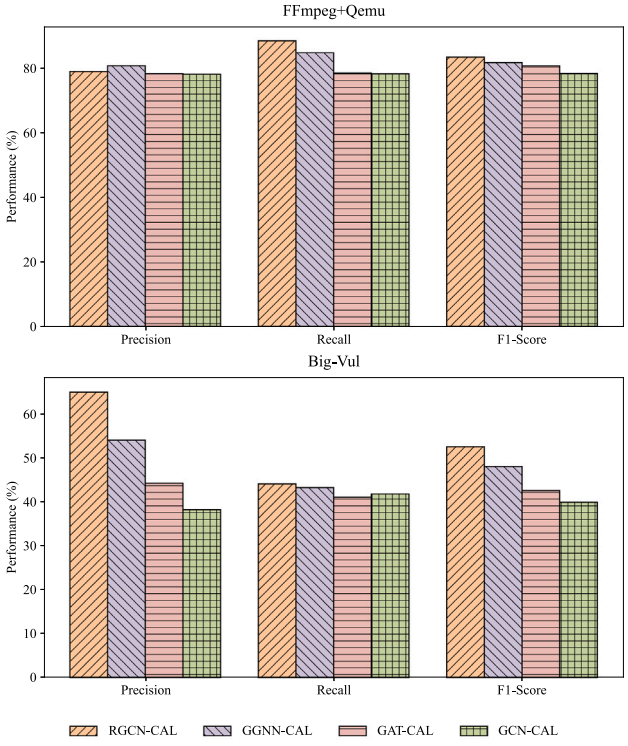

Fig. 6. Performance of RGCN-CAL module variants with different GNNs on the FFmpeg+Qemu and Big-Vul dataset.

Still, its lack of native support for encoding edge types undermines its performance in the multi-relational context of ACFGs, where the semantics of relations are essential for accurate vulnerability detection. By comparison, GCN (Kipf and Welling, 2016) shows the weakest performance among the evaluated models. Its simplistic architecture is poorly suited for ACFGs, as it fails to account for the diversity of relationships between code statement nodes which is a crucial factor for modeling complex graphs.

> **Answer to RQ3.1:** The RGCN-based model outperforms others by effectively capturing diverse relational structures, whereas GGNN, GAT, and GCN show limitations in handling the relational semantics of ACFG.

5.3.2. Effectiveness of individual loss functions

In the task of vulnerability detection, the effective extraction of causal features and the suppression of spurious features are crucial for improving model performance. The VulDIAC model achieves this through the synergistic design of multiple loss functions, including $\mathcal{L}_{\text{sup}}$, $\mathcal{L}_{\text{unif}}$, $\mathcal{L}_{\text{caus}}$, and $\mathcal{L}_{\text{contras}}$. To investigate the independent effects and collaborative contributions of these loss functions during model optimization, we conducted ablation experiments by removing specific loss terms. We tracked the variant of $\mathcal{L}_{\text{sup}}$ during training for different

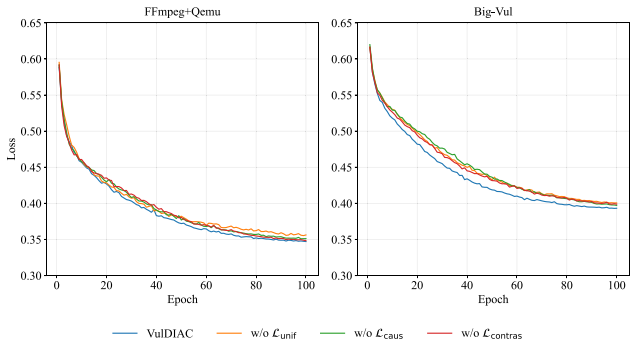

Fig. 7. Comparison of training loss curves for loss function variants over epochs on the FFmpeg+Qemu and Big-Vul dataset.

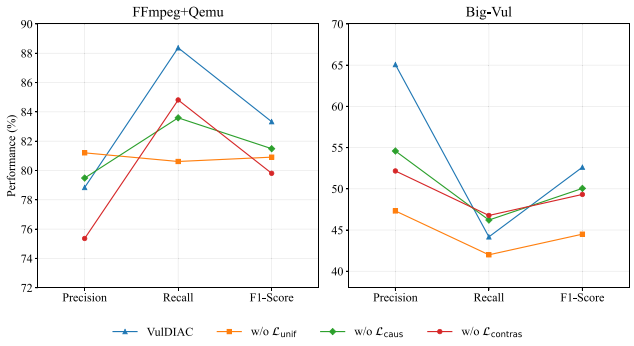

Fig. 8. Performance comparison of loss function variants on the FFmpeg+Qemu and Big-Vul dataset.

model variants, as $\mathcal{L}_{sup}$ is responsible for the classification and directly reflects vulnerability detection performance.

As shown in Fig. 7, the VulDIAC model exhibits the fastest convergence rate and the lowest final loss on both the FFmpeg+Qemu and Big-Vul datasets. This demonstrates that the synergy among loss functions significantly enhances optimization efficiency. The loss decreases more slowly and ultimately reaches a higher value on the Big-Vul dataset, indicating that the imbalanced sample distribution in this dataset harms the model training. For the model without $\mathcal{L}_{unif}$, both datasets show similar initial convergence rates as VulDIAC, but with higher final losses, suggesting that the uniformity constraint on spurious features plays a cross-dataset, universal role in optimizing stability. Meanwhile, the models without $\mathcal{L}_{caus}$ or $\mathcal{L}_{contras}$ exhibit varying degrees of performance degradation. Notably, the absence of $\mathcal{L}_{caus}$ causes more pronounced loss fluctuations on the Big-Vul dataset, highlighting the importance of causal feature learning for robustness on complex datasets. The absence of $\mathcal{L}_{contras}$ leads to slower early convergence on both datasets, further validating the essential role of contrastive learning in feature decoupling.

Fig. 8 provides further performance evaluation on both datasets. For the FFmpeg+Qemu dataset, the VulDIAC model outperforms all other variants in both recall and F1-Score, particularly showing a significant advantage in recall. This demonstrates that VulDIAC effectively captures causal features, avoids interference from spurious features, and identifies more vulnerable functions. However, for the Big-Vul dataset, the performance advantage of VulDIAC is less pronounced, especially in recall. In the scenario of large-scale, imbalanced datasets, the effectiveness of multi-task loss learning in covering more potential vulnerabilities is limited. However, VulDIAC still achieves higher precision and F1-Score, proving it to be a more robust model. Additionally, the variant without $\mathcal{L}_{unif}$ shows the worst performance on the Big-Vul dataset. While it achieves relatively good precision on the FFmpeg+Qemu dataset, it suffers a notable drop in recall,

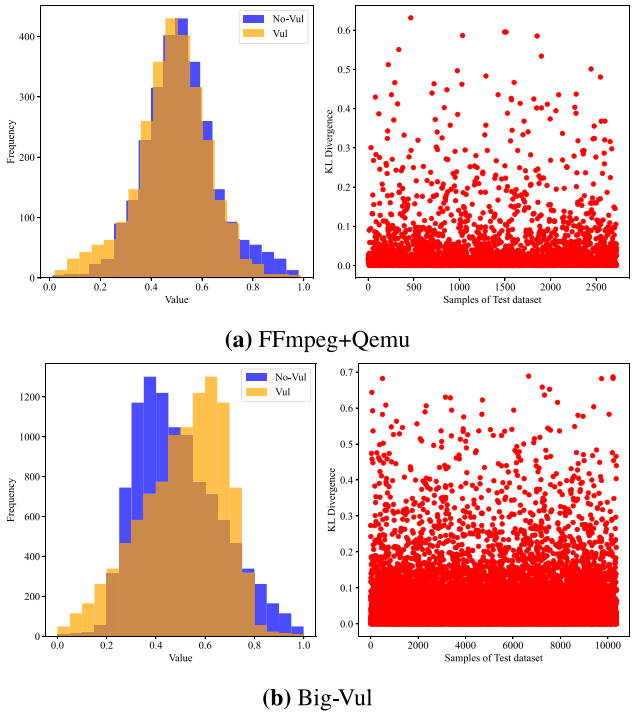

Fig. 9. Distribution of z_s (left) and the KL divergence between z_s and uniform distribution (right) on the FFmpeg+Qemu and Big-Vul dataset.

suggesting that the model may prioritize more precise predictions for non-vulnerable samples. On the other hand, the variants without $\mathcal{L}_{caus}$ and without $\mathcal{L}_{contras}$ experience a sharp drop in F1-Score on both datasets, emphasizing the indispensable roles of causal feature learning and contrastive learning.

> **Answer to RQ3.2:** The design of the multi-task loss enables VulDIAC to effectively extract causal features and avoid reliance on spurious features, which is essential for accurate vulnerability detection.

5.4. Effectiveness of uniform distribution constraints

In this section, we investigate whether applying KL divergence can effectively constrain the model's predictions, influenced by spurious features, to a uniform distribution and reduce the mutual information between these spurious features and vulnerability labels. To this end, we analyze the predicted probability distributions of z_s (as defined in Eq. (11).) on the testing sets of the FFmpeg+Qemu (Zhou et al., 2019) and Big-Vul (Fan et al., 2020) datasets. As shown in Fig. 9, we report the distribution of z_s and the corresponding KL divergence values with respect to the uniform distribution.

For the FFmpeg+Qemu dataset, as depicted in Fig. 9(a), the distribution of z_s is relatively well-constrained, with most values falling between 0.4 and 0.6. The low KL divergence values further support this, with 87.71% of the samples having a KL divergence less than 0.1 and 94.56% having a KL divergence less than 0.2. A smaller KL divergence indicates a closer match between the model's prediction distribution and the uniform distribution, suggesting that z_s is approximately uniformly distributed. It indicates that the mutual information between spurious features and labels has been reduced, and our uniformity constraint is effective. On the other hand, for the more complex and class-imbalanced Big-Vul dataset, as depicted in Fig. 9(b), the values of z_s are primarily distributed between 0.3 and 0.7, indicating a wider

Table 4
Comparison of function-level vulnerability detection performance of LineVD, LineVul, and VulDIAC on two datasets.

Method	FFmpeg+Qemu				Big-Vul			
	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
LineVD	–	–	–	–	88.93	38.51	30.63	34.12
LineVul	65.97	61.25	68.73	64.77	**98.11**	**92.69**	**89.50**	**91.07**
VulDIAC	**83.47**	**78.84**	**88.37**	**83.33**	95.99	65.07	44.17	52.62

range compared to FFmpeg+Qemu. This suggests that the constraint on spurious features is less stable. Nevertheless, 80.93% of the samples have a KL divergence less than 0.1, and 93.76% have a KL divergence less than 0.2, demonstrating that the model's prediction distribution still maintains a relatively small difference from the uniform distribution, although not as effectively as observed in the FFmpeg+Qemu dataset. We attribute this difference to the class imbalance present in the Big-Vul dataset, which introduces bias and reduces the effectiveness of the constraint on spurious features.

Answer to RQ4: Optimizing KL divergence effectively constrains the model's predictions based on spurious features to a uniform distribution, reducing mutual information between spurious features and labels. This effect is more pronounced on the balanced FFmpeg+Qemu dataset.

5.5. Line-level vulnerability localization

In this section, we first compare the performance of LineVD (Hin et al., 2022), LineVul (Fu and Tantithamthavorn, 2022), and VulDIAC on function-level vulnerability detection tasks using the FFmpeg+Qemu (Zhou et al., 2019) and Big-Vul (Fan et al., 2020) datasets. Specifically, the objective (loss) of LineVD learns from both function-level and statement-level features, enabling it to predict whether specific function statements are vulnerable. This requires more fine-grained line-level labels. However, since the FFmpeg+Qemu dataset does not provide line-level vulnerability labels, LineVD does not apply to this dataset, as indicated by the '-' in Table 4. Therefore, we compare the Top-k accuracy of line-level vulnerability detection performance on the Big-Vul dataset, as it obtains line-level vulnerability ground-truth labels.

As illustrated in Table 4, VulDIAC outperforms LineVul in all evaluation metrics on the FFmpeg+Qemu dataset, demonstrating its competitive capacity in vulnerability detection. However, for the Big-Vul dataset, while VulDIAC performs better than LineVD, LineVul exhibits the best performance. This is because LineVul was pre-trained on the Big-Vul dataset using the Byte Pair Encoding (BPE) (Sennrich et al., 2016) subword tokenizer, which allows it to learn the relationships between tokens better and generate more meaningful vector representations. Additionally, LineVul's training process includes fine-tuning, which further enhances its vulnerability classification performance on the Big-Vul dataset.

To evaluate the line-level vulnerability localization capability of VulDIAC, we designed two causal attention-based scoring mechanisms: VulDIAC-NA and VulDIAC. VulDIAC-NA relies solely on node attention for line-level prediction, whereas VulDIAC, as described in Eq. (16), integrates both node and edge attention through a symmetric propagation mechanism. For functions identified as vulnerable, code lines with higher scores are considered to be high-risk potential vulnerability locations. For example, if vulnerable lines appear in the Top-5 or Top-10, it suggests that the model has accounted for the root causes of the vulnerability in its predictions, indicating that the causal features effectively capture these root causes and that causal learning is operating as expected. Conversely, if no vulnerable lines are found in either the Top-5 or Top-10, it suggests that the model has failed to identify the fundamental causes of these vulnerabilities, and causal learning has not been effective for these cases. This implies that the current model has limitations in capturing the complex vulnerability patterns within certain vulnerable functions.

Table 5
Top-k accuracy comparison on the Big-Vul dataset.

Method	Top-5 Acc	Top-10 Acc
LineVD	**76.54**	**83.35**
LineVul	44.48	59.12
VulDIAC-NA	35.62	52.15
VulDIAC	46.37	61.36

As shown in Table 5, VulDIAC outperforms LineVul in both Top-5 and Top-10 accuracy metrics. Specifically, compared to LineVul, VulDIAC's Top-5 accuracy improved by 4.25%, and its Top-10 accuracy improved by 3.79%. Furthermore, VulDIAC performs significantly better than VulDIAC-NA, with a 30.18% increase in Top-5 accuracy and a 17.66% increase in Top-10 accuracy. However, the performance of VulDIAC-NA is inferior to LineVul, which highlights the importance of edge attention in improving line-level vulnerability localization accuracy. Nevertheless, both VulDIAC and LineVul fall short of LineVD in terms of Top-k accuracy. Both LineVul and VulDIAC calculate vulnerability scores for code lines based on the model's internal attention mechanisms. LineVul uses the multi-head attention mechanism in the Transformer model to aggregate attention scores and identify the most likely vulnerable code lines. In contrast, VulDIAC utilizes the sum of attention scores from both nodes and adjacent edges in the causal attention module to predict vulnerable lines. On the other hand, LineVD employs two levels of objective functions to classify vulnerabilities at both the function and line levels. The line-level vulnerability classification loss allows LineVD to learn in finer detail whether a specific code line is vulnerable, resulting in higher Top-5 and Top-10 accuracy. Specifically, LineVD outperforms VulDIAC by 65.06% in Top-5 accuracy and by 35.84% in Top-10 accuracy. However, in function-level detection, VulDIAC still performs well compared to LineVD, demonstrating the competitive strength of our approach.

Answer to RQ5: VulDIAC enhances the ability to locate vulnerabilities by leveraging causal node and edge attention, achieving better Top-k accuracy compared to LineVul. However, it does not achieve the same level of performance as LineVD, which implements line-level vulnerability classification.

6. Discussion

6.1. Why VulDIAC effective?

We attribute the effectiveness of VulDIAC to two key aspects. First, we incorporate semantic information such as program execution path dependencies and variable define-use relations into the ACFG, addressing the limitations of CFGs in modeling semantic dependencies within code. Second, we leverage the multi-task based RGCN-CAL module to perform feature representation on the multi-relational ACFG, effectively capturing the semantic distinctions among different relationships. Furthermore, our causal attention mechanism, guided by causal

```
1 void kvm_lapic_sync_to_vapic(struct kvm_vcpu *vcpu){
2    u32 data, tpr;
3    int max_irr, max_isr;
4    struct kvm_lapic *apic = vcpu->arch.apic;
5   -void *vapic;
6
7    apic_sync_pv_eoi_to_guest(vcpu, apic);
8
9    if (!test_bit(KVM_APIC_CHECK_VAPIC, &vcpu->arch.apic_attention))
10       return;
>    [···]
21  -vapic = kmap_atomic(vcpu->arch.apic->vapic_page);
22  -*(u32 *)(vapic + offset_in_page(vcpu->arch.apic->vapic_addr)) = data;
23  -kunmap_atomic(vapic);
    +kvm_write_guest_cached(vcpu->kvm, &vcpu->arch.apic->vapic_cache, &data,
                sizeof(u32));
24 }
```

Ground truth vulnerable lines:	Rank scores:[5.10, **3.98**, 2.57, **2.38**, 2.34]
[5, 21, **22**, **23**]	Top-5 lines:[20, **23**, 1, **22**, 13]

Fig. 10. A case study for interpretable line-level vulnerability localization on CVE-2013-6368.

inference (Sui et al., 2022, 2024), enables the model to focus on causal features for vulnerability detection, which reduces the interference of spurious features. This approach also contributes to the performance in providing interpretable line-level vulnerability interpretations.

6.2. Interpretable case

We perform a visualized interpretability case on CVE-2013-6368 (CVE-2013-6368, 2013), as shown in Fig. 10. In this example, the red-highlighted lines (5, 21, 22, and 23) represent vulnerable code, while the green-highlighted lines correspond to the patched code. The vulnerability arises because the code does not validate whether the pointer is valid. If *vapic_page* or *vapic_addr* is a NULL pointer, it leads to a null pointer dereference. Additionally, if *vapic_addr* contains a maliciously crafted offset, it may result in unauthorized writes to kernel memory.

VulDIAC identifies this vulnerability by detecting its presence and subsequently employs causal attention mechanisms to aggregate node-level and edge-level attention scores, which are then used to compute vulnerability scores for each line of code. The scores are ranked in descending order, and the corresponding lines are prioritized. In this case, lines 23 and 22 are ranked second and fourth, respectively, for their association with the vulnerability. This case study demonstrates that VulDIAC can capture vulnerability patterns in source code, locate high-risk code segments, and provide interpretable line-level explanations. It offers clear line-level insights to developers and security analysts, helping them quickly identify the root causes of vulnerabilities.

6.3. Threats to validity

Internal validity. A significant threat to the validity of vulnerability datasets arises from inherent data quality issues. As noted by Croft et al. (2023), these datasets are particularly susceptible to duplicates and label noise. Functions are often automatically labeled based on vulnerability-fixing commits in large-scale datasets, and due to ambiguities in security patches, the accuracy of this labeling is typically lower compared to manual annotation (Ding et al., 2025). Furthermore, duplicate functions undermine the evaluation of model generalization capabilities. To address these issues, we filtered the FFmpeg+Qemu (Zhou et al., 2019) and Big-Vul (Fan et al., 2020) datasets by eliminating duplicates, parsing errors by Joern (Yamaguchi et al., 2014), and functions with fewer than three ACFG nodes. The original Big-Vul dataset contained 188,636 functions, while the FFmpeg+Qemu dataset included 27,318. Following the sanitization process, their sizes were reduced to 103,729 and 27,237, respectively, as presented in Table 1. However, residual risks remain despite these measures, particularly from interference caused by semantically similar functions and imperfect labels. These factors may still impair the model's ability to discern subtle vulnerability patterns, potentially compromising its generalization performance in real-world scenarios. In addition, some functions from FFmpeg+Qemu are included in Big-Vul. Although the model evaluations are based on separately trained models, the data overlap may introduce potential domain biases that impact the independence of the experimental evaluation. On the other hand, the dataset differences mentioned above caused a deviation between our reproduced results on the baseline. Although the same 8:1:1 train–validation–test split was applied, the differences in dataset size and composition led to discrepancies in the results. For instance, LineVul (Fu and Tantithamthavorn, 2022) claims a Top-10 accuracy of 65% and our reproduced result was 59.12%.

Although our method demonstrates promising results in the current experiments, there are still potential threats that could affect the model's effectiveness. The black-box nature of deep learning models presents certain limitations, particularly as VulDIAC does not provide a transparent mechanism to reveal how features are selected. Consequently, it is impossible to guarantee that causal features are used as expected. Nevertheless, we leverage its attention mechanism to analyze which features the model predominantly focuses on. Based on this analysis, we provide line-level vulnerability explanations to verify the effective use of causal features, as demonstrated by the experimental analysis in section 5.5 and the interpretable case study in section 6.2. As for code embedding module, we utilize the widely used CodeBert (Feng et al., 2020), which has shown stability across various tasks. However, other advanced embedding models, such as CodeT5 (Wang et al., 2021a) and UniXcoder (Guo et al., 2022), may prove to be more effective for our approach. In future work, we will comprehensively study the impact of different code language models on vulnerability detection models, especially large language models (LLMs). Additionally, we set fixed values for the multi-task loss parameters, as adjusting λ_1, λ_2, and λ_3 is intended to align with the specific characteristics of particular domains or datasets. Future research could investigate strategies for adaptively tuning the weight parameters of each loss term to enhance generalization across diverse scenarios.

External validity. Our research is focused on C/C++ datasets, which may restrict the applicability of VulDIAC to specific program languages. Nonetheless, VulDIAC is theoretically extensible and can be adapted to other program languages by modifying the static analysis tools used for ACFG construction. However, the differences in syntax and program paradigms between languages could affect the model's accuracy and performance. Cross-language learning in deep learning is generally considered a cross-domain problem, which is inherently more complex and less predictable than cross-dataset or cross-project learning within the same language. These factors will also pose challenges to the interpretability of VulDIAC, particularly when applied across different programming languages.

7. Related work

7.1. Learning-based vulnerability detection

Learning-based vulnerability detection methods have emerged as a response to the limitations of traditional analysis techniques and have become a significant area of research in software engineering in recent years. Some approaches (Russell et al., 2018; Alon et al., 2019) represent code as flat token sequences and employ sequence-based analysis techniques to extract features and train neural networks for vulnerability detection. For instance, Russell et al. (2018) tokenized source code into lexical units (e.g., keywords, variables, operators) and trained an RNN to automatically extract features that distinguish vulnerable functions from non-vulnerable ones. Similarly, Li et al. introduced VulDeePecker (Li et al., 2018) and SySeVR (Li et al., 2022b), which use static analysis techniques to extract source code slices based on potential vulnerability factors (e.g., API calls, memory accesses).

These slices are treated as token sequences, and RNNs are employed to capture code features. However, token-based models often fail to preserve the structural and semantic information of the code.

In recent years, graph-based vulnerability detection methods (Zhou et al., 2019; Chakraborty et al., 2022; Cao et al., 2021) have been proposed, achieving state-of-the-art performance. These approaches typically leverage static analysis techniques to represent source code as structured graphs. For example, Devign (Zhou et al., 2019) and Reveal (Chakraborty et al., 2022) utilize GGNN (Li et al., 2015) to extract features from multi-relational CPGs, capturing data and control dependencies. Despite their effectiveness, GGNN-based methods have certain limitations. BGNN4VD (Cao et al., 2021) incorporates bidirectional graph neural networks to integrate both syntactic and semantic information of the code, significantly improving vulnerability detection performance. FUNDED (Wang et al., 2021b) focuses on function-level vulnerability detection by modeling enhanced multi-relational ASTs. Wen et al. introduced two approaches, AMPLE (Wen et al., 2023) and MAGNET (Wen et al., 2024), which address different aspects of CPGs for vulnerability detection. AMPLE reduces the complexity of AST nodes in CPGs and leverages edge-aware attention-based graph convolutional networks to identify vulnerabilities. MAGNET focuses on modeling metapaths to represent the multi-relational and heterogeneous properties of CPGs, utilizing heterogeneous attention networks (HAN) for function-level vulnerability detection. While both methods demonstrate strong performance, AST-based techniques often encounter challenges stemming from the high density of nodes, which can lead to increased noise and difficulties in capturing long-range dependencies. With this context, IVDetect (Li et al., 2021) improves vulnerability detection by leveraging features extracted from PDGs. Slicing techniques have been widely utilized in approaches such as DeepWukong (Cheng et al., 2021), mVulPreter (Zou et al., 2022), and SlicedLocator (Wu et al., 2023) to retrieve essential information from PDGs. These methods pinpoint potential vulnerability locations within programs and generate forward and backward slices based on data and control dependency edges, subsequently training GNNs for slice-level vulnerability detection. Alternatively, VulCNN (Wu et al., 2022) adopts a distinct approach by modeling PDG node features using three centrality measures, projecting them into separate feature channels, and applying convolutional neural networks (CNNs) (Kim, 2014) to extract features, achieving strong performance in vulnerability detection tasks.

In addition, CFG serves as a fundamental structure in many static analysis tools, effectively capturing program control flow logic and representing potential execution paths. EPVD (Zhang et al., 2023) encodes specific execution paths within CFGs to represent code fragments. However, the absence of data flow information in CFGs restricts their capacity to comprehensively model vulnerability patterns. In this study, we enhance CFG and incorporate reaching definition analysis to enable ACFG to effectively represent data usage patterns that may related to vulnerabilities. We also expand the representation of control relationships along program execution paths by introducing dominance relationships, offering a more detailed depiction of program behavior.

7.2. Interpretable vulnerability localization

The requirement for explainability in vulnerability detection methods is particularly critical, as relying solely on binary classification results (e.g., vulnerable or non-vulnerable) fails to establish trust (Warnecke et al., 2020). However, the inherently black-box nature of GNN models makes it challenging to provide clear and interpretable evidence for classifying code samples (Steenhoek et al., 2023; Hu et al., 2023). In response to this limitation, IVDetect (Li et al., 2021) employs GNNExplainer (Ying et al., 2019) to extract the minimal PDG subgraph necessary to explain vulnerabilities while maintaining the original model's prediction. This approach offers a model-agnostic interpretation method. In contrast, model-internal interpretation techniques rely directly on the intrinsic properties of the model itself, eliminating the need for additional external explainers. For example, LineVD (Hin et al., 2022) utilizes GAT (Veličković et al., 2017) to predict suspected vulnerabilities by using objective function at the statement level to predict whether a statement is vulnerable. Similarly, mVulPreter (Zou et al., 2022) focuses on essential slices by filtering out those with low relevance to vulnerabilities and provides interpretation by aggregating the attention scores of code lines within the retained slices. LineVul (Fu and Tantithamthavorn, 2022) employs the pre-trained large language model (LLM) (Feng et al., 2020) to learn code semantics, encoding features through bidirectional self-attention mechanisms (Vaswani, 2017) in Transformer layers and combining attention scores from multiple Transformer layers to achieve line-level interpretation. While VulDIAC also adopts a model-internal interpretation approach, it distinguishes itself by prioritizing the causal reasoning process within the model's decision-making. The model incorporates a causal attention mechanism to assign importance to nodes and edges, enabling interpretability.

7.3. Causal inference

Recently, Cito et al. (2022) explore methods for generating counterfactual explanations for machine learning models applied to source code, which help developers better understand the model's decision-making process. Causal inference theory (Sui et al., 2022, 2024; Pearl, 2009), as an analytical framework, empowers deep learning models to systematically identify and exclude potential confounding factors, enabling a more precise elucidation of the causal relationships between predictions and their latent representations. In the field of vulnerability detection, Rahman et al. (2024) are the first to reveal that deep learning-based vulnerability detection models rely on spurious features, such as variable identifiers and API names. By leveraging explicit causal learning techniques, such as do-calculus, they systematically eliminate the impact of these spurious features on model predictions. Coca (Cao et al., 2024) introduce a dual-perspective causal inference methodology that integrates factual and counterfactual explanations, offering pivotal insights into code statements that play a decisive role in identifying vulnerabilities within other detection models. Our work neither concentrates on identifying spurious features nor seeks to explain prediction errors directly. Instead, we introduce an innovative strategy that implicitly steers the model toward learning causal patterns of vulnerabilities by employing attention mechanisms.

8. Conclusion

This paper presents the VulDIAC framework for vulnerability detection, which delivers notable advancements in both detection accuracy and interpretability by leveraging an augmented control flow graph (ACFG) and a purpose-built multi-task causal learning module. The augmented CFG is specifically engineered to better capture program execution path dependencies and define-use relationships within the data flow, addressing the inherent limitations of traditional methods in representing the semantic structure of source code. To effectively exploit the multi-relational characteristics of the augmented CFG, a multi-relational graph convolutional neural network is employed, facilitating the extraction of rich and meaningful semantic features underlying each type of relationship. Furthermore, we frame the vulnerability detection task from the perspective of causal learning. By incorporating multiple loss functions, we effectively mitigate the influence of spurious features on model predictions, resulting in more robust feature representations and interpretable explanations. VulDIAC substantially improves F1-Score, with increases of 27.16% and 53.59% on two benchmark datasets, respectively. Additionally, it demonstrates better performance in Top-k accuracy compared to LineVul, underscoring its effectiveness in vulnerability interpretation. In the future, we will further explore adaptive loss weight adjustment mechanisms to improve the model's adaptability across diverse scenarios. Moreover, we plan to conduct causal interventions at the data level to reveal more transparent causal vulnerability patterns and provide more reliable interpretations for software vulnerabilities.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This study was supported by the National Natural Science Foundation of China (no.62376240) and the S&T Program of Hebei (nos.236Z0702G, 226Z0701G, 236Z0304G).

References

Alfred, V.A., Monica, S.L., Jeffrey, D.U., 2007. Compilers Principles, Techniques & Tools. Pearson Education.

Allamanis, M., Barr, E.T., Devanbu, P., Sutton, C., 2018. A survey of machine learning for big code and naturalness. ACM Comput. Surv. 51 (4), 1–37.

Alon, U., Brody, S., Levy, O., Yahav, E., 2019. Code2seq: Generating sequences from structured representations of code. arXiv:1808.01400. URL https://arxiv.org/abs/1808.01400.

Booth, H., Rike, D., Witte, G.A., 2013. The National Vulnerability Database (nvd): Overview. Harold Booth, Doug Rike, Gregory A. Witte.

Cao, S., Sun, X., Bo, L., Wei, Y., Li, B., 2021. BGNN4VD: Constructing bidirectional graph neural-network for vulnerability detection. Inf. Softw. Technol. 136, 106576.

Cao, S., Sun, X., Wu, X., Lo, D., Bo, L., Li, B., Liu, W., 2024. Coca: Improving and explaining graph neural network-based vulnerability detection systems. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ACM, Lisbon Portugal, pp. 1–13.

Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2022. Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9), 3280–3296.

Cheng, X., Wang, H., Hua, J., Xu, G., Sui, Y., 2021. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. ACM Trans. Softw. Eng. Methodol. (TOSEM) 30 (3), 1–33.

Cito, J., Dillig, I., Murali, V., Chandra, S., 2022. Counterfactual explanations for models of code. In: Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice. pp. 125–134.

Croft, R., Babar, M.A., Kholoosi, M.M., 2023. Data quality for software vulnerability datasets. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 121–133.

CVE, Common vulnerabilities and exposures, https://cve.mitre.org/.

CVE-2013-6368, 2013. https://www.cvedetails.com/cve/CVE-2013-6368.

CVE-2016-4002, 2016. https://www.cvedetails.com/cve/CVE-2016-4002.

Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., Chen, Y., 2025. Vulnerability detection with code language models: How far are we? In: 2025 IEEE/ACM 47th International Conference on Software Engineering. ICSE, IEEE Computer Society, pp. 1729–1741.

Fan, J., Li, Y., Wang, S., Nguyen, T.N., 2020. AC/C++ code vulnerability dataset with code changes and CVE summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories. pp. 508–512.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020. Codebert: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155.

Ferrante, J., Ottenstein, K.J., Warren, J.D., 1987. The program dependence graph and its use in optimization. ACM Trans. Program. Lang. Syst. (TOPLAS) 9 (3), 319–349.

FFmpeg, http://www.ffmpeg.org/.

Fu, M., Tantithamthavorn, C., 2022. Linevul: A transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories. pp. 608–620.

Gan, S., Zhang, C., Qin, X., Tu, X., Li, K., Pei, Z., Chen, Z., 2022. Path sensitive fuzzing for native applications. IEEE Trans. Dependable Secur. Comput. 19 (3), 1544–1561.

Ghaffarian, S.M., Shahriari, H.R., 2017. Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey. ACM Comput. Surv. 50 (4), 1–36.

Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J., 2022. Unixcoder: Unified cross-modal pre-training for code representation. arXiv preprint arXiv:2203.03850.

Hin, D., Kan, A., Chen, H., Babar, M.A., 2022. LineVD: statement-level vulnerability detection using graph neural networks. In: Proceedings of the 19th International Conference on Mining Software Repositories. pp. 596–607.

Hu, Y., Wang, S., Li, W., Peng, J., Wu, Y., Zou, D., Jin, H., 2023. Interpreters for GNN-based vulnerability detection: Are we there yet? In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. In: ISSTA 2023, Association for Computing Machinery, New York, NY, USA, pp. 1407–1419.

Jang, J., Agrawal, A., Brumley, D., 2012. ReDeBug: Finding unpatched code clones in entire OS distributions. In: 2012 IEEE Symposium on Security and Privacy. pp. 48–62.

Kim, Y., 2014. Convolutional neural networks for sentence classification. In: Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, a Meeting of SIGDAT, a Special Interest Group of the ACL. ACL, pp. 1746–1751.

Kim, S., Woo, S., Lee, H., Oh, H., 2017. VUDDY: A scalable approach for vulnerable code clone discovery. In: 2017 IEEE Symposium on Security and Privacy. SP, pp. 595–614.

Kipf, T.N., Welling, M., 2016. Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.

Lee, M., Cho, S., Jang, C., Park, H., Choi, E., 2006. A rule-based security auditing tool for software vulnerability detection. In: 2006 International Conference on Hybrid Information Technology, 2, IEEE, pp. 505–512.

Li, Y., Tarlow, D., Brockschmidt, M., Zemel, R., 2015. Gated graph sequence neural networks. arXiv preprint arXiv:1511.05493.

Li, Y., Wang, S., Nguyen, T.N., 2021. Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 292–303.

Li, Z., Zou, D., Xu, S., Chen, Z., Zhu, Y., Jin, H., 2022a. Vuldeelocator: a deep learning-based fine-grained vulnerability detector. IEEE Trans. Dependable Secur. Comput. 19 (4), 2821–2837.

Li, Z., Zou, D., Xu, S., Jin, H., Qi, H., Hu, J., 2016. VulPecker: an automated vulnerability detection system based on code similarity analysis. In: Proceedings of the 32nd Annual Conference on Computer Security Applications. ACSAC '16, Association for Computing Machinery, New York, NY, USA, pp. 201–213.

Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2022b. Sysevr: A framework for using deep learning to detect software vulnerabilities. IEEE Trans. Dependable Secur. Comput. 19 (4), 2244–2258.

Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. arXiv preprint arXiv:1801.01681.

Liang, C., Wei, Q., Du, J., Wang, Y., Jiang, Z., 2025. Survey of source code vulnerability analysis based on deep learning. Comput. Secur. 148, 104098.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019. Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692.

Pearl, J., 2009. Causality: Models, Reasoning and Inference, second ed. Cambridge University Press, USA.

Pearl, J., 2014. Interpretation and identification of causal mediation. Psychol. Methods 19 (4), 459.

Pham, V.-T., Böhme, M., Santosa, A.E., Căciulescu, A.R., Roychoudhury, A., 2021. Smart greybox fuzzing. IEEE Trans. Softw. Eng. 47 (9), 1980–1997.

Qemu, https://www.qemu.org/.

Rahman, M.M., Ceka, I., Mao, C., Chakraborty, S., Ray, B., Le, W., 2024. Towards causal deep learning for vulnerability detection. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ACM, Lisbon Portugal, pp. 1–11.

Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M., 2018. Automated vulnerability detection in source code using deep representation learning. In: 2018 17th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 757–762.

Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G., 2008. The graph neural network model. IEEE Trans. Neural Netw. 20 (1), 61–80.

Schlichtkrull, M., Kipf, T.N., Bloem, P., Berg, R.v.d., Titov, I., Welling, M., 2017. Modeling relational data with graph convolutional networks. arXiv preprint arXiv:1703.06103.

Sennrich, R., Haddow, B., Birch, A., 2016. Neural machine translation of rare words with subword units. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics. ACL, Association for Computational Linguistics, pp. 1715–1725.

Steenhoek, B., Rahman, M.M., Jiles, R., Le, W., 2023. An empirical study of deep learning models for vulnerability detection. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 2237–2248.

Sui, Y., Mao, W., Wang, S., Wang, X., Wu, J., He, X., Chua, T.-S., 2024. Enhancing out-of-distribution generalization on graphs via causal attention learning. ACM Trans. Knowl. Discov. Data 18 (5), 1–24.

Sui, Y., Wang, X., Wu, J., Lin, M., He, X., Chua, T.-S., 2022. Causal attention for interpretable and generalizable graph classification. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 1696–1705.

Suter, B.W., 1990. The multilayer perceptron as an approximation to a Bayes optimal discriminant function. IEEE Trans. Neural Netw. 1 (4), 291.

Vaswani, A., 2017. Attention is all you need. Adv. Neural Inf. Process. Syst. 30.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., 2017. Graph attention networks. arXiv preprint arXiv:1710.10903.

Wang, Y., Wang, W., Joty, S., Hoi, S.C., 2021a. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. arXiv preprint arXiv:2109.00859.

Wang, H., Ye, G., Tang, Z., Tan, S.H., Huang, S., Fang, D., Feng, Y., Bian, L., Wang, Z., 2021b. Combining graph-based learning with automated data collection for code vulnerability detection. IEEE Trans. Inf. Forensics Secur. 16, 1943–1958.

Warnecke, A., Arp, D., Wressnegger, C., Rieck, K., 2020. Evaluating explanation methods for deep learning in security. In: 2020 IEEE European Symposium on Security and Privacy. EuroS&P, IEEE, pp. 158–174.

Wen, X.-C., Chen, Y., Gao, C., Zhang, H., Zhang, J.M., Liao, Q., 2023. Vulnerability detection with graph simplification and enhanced graph representation learning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering. ICSE, IEEE, pp. 2275–2286.

Wen, X.-C., Gao, C., Ye, J., Li, Y., Tian, Z., Jia, Y., Wang, X., 2024. Meta-path based attentional graph learning model for vulnerability detection. IEEE Trans. Softw. Eng. 50 (3), 360–375.

Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., Jin, H., 2022. VulCNN: An image-inspired scalable vulnerability detection system. In: Proceedings of the 44th International Conference on Software Engineering. pp. 2365–2376.

Wu, B., Zou, F., Yi, P., Wu, Y., Zhang, L., 2023. SlicedLocator: Code vulnerability locator based on sliced dependence graph. Comput. Secur. 134, 103469.

Yamaguchi, F., Golde, N., Arp, D., Rieck, K., 2014. Modeling and discovering vulnerabilities with code property graphs. In: 2014 IEEE Symposium on Security and Privacy. IEEE, pp. 590–604.

Ying, Z., Bourgeois, D., You, J., Zitnik, M., Leskovec, J., 2019. Gnnexplainer: Generating explanations for graph neural networks. Adv. Neural Inf. Process. Syst. 32.

Zhang, J., Liu, Z., Hu, X., Xia, X., Li, S., 2023. Vulnerability detection by learning from syntax-based execution paths of code. IEEE Trans. Softw. Eng. 49 (8), 4196–4212.

Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. Adv. Neural Inf. Process. Syst. 32.

Zou, D., Hu, Y., Li, W., Wu, Y., Zhao, H., Jin, H., 2022. Mvulpreter: A multi-granularity vulnerability detection system with interpretations. IEEE Trans. Dependable Secur. Comput. 1–12.

Zou, D., Wang, S., Xu, S., Li, Z., Jin, H., 2021. μVulDeePecker: A deep learning-based system for multiclass vulnerability detection. IEEE Trans. Dependable Secur. Comput. 18 (5), 2224–2236.